

Laboratoire réseau virtualisé et automatisation de sa surveillance

Routage OSPF et BGP · containerlab · FRRouting · Python / Netmiko

Auteur	Lodric
Formation	EFREI Paris
Nature	Projet personnel d'apprentissage
Date	26 septembre 2026
Version	1.0

Sommaire

1. Synthèse.....	3
2. Contexte et objectifs.....	4
2.1 Point de départ.....	4
2.2 Objectifs.....	4
2.3 Contraintes.....	4
2.4 Démarche et outils d'assistance.....	4
3. Les notions clés, expliquées simplement.....	6
3.1 Réseau, adresse IP et sous-réseau.....	6
3.2 Routeur et table de routage.....	6
3.3 Routage dynamique et OSPF.....	6
3.4 BGP et systèmes autonomes.....	7
3.5 Virtualisation, WSL2 et conteneurs.....	7
3.6 containerlab et FRRouting.....	7
3.7 SSH, automatisation et dérive de configuration.....	8
4. Architecture du laboratoire.....	9
4.1 Vue d'ensemble technique.....	9
4.2 Topologie du réseau.....	9
4.3 Plan d'adressage.....	10
4.4 Logique de routage et justification des choix.....	11
4.5 Organisation des fichiers du projet.....	11
5. Choix techniques et alternatives étudiées.....	12
5.1 Plateforme d'exécution.....	12
5.2 Outil de simulation réseau.....	12
5.3 Logiciel des routeurs.....	12
5.4 Bibliothèque d'automatisation.....	13
6. Déroulement du projet.....	14
6.1 Étape 1 : conception.....	14
6.2 Étape 2 : validation préalable en environnement de test.....	14
6.3 Étape 3 : préparation du poste de travail.....	15
6.4 Étape 4 : déploiement réel et tests.....	15
6.5 Étape 5 : remise à zéro et vérification manuelle.....	15
7. Résultats : le routage (phase 1).....	16
7.1 Voisinages OSPF.....	16
7.2 Table de routage de R1.....	16
7.3 Session BGP entre les deux organisations.....	16
7.4 Test de bout en bout.....	17
8. Résultats : l'automatisation (phase 2).....	18
8.1 Principe de fonctionnement.....	18
8.2 Les trois outils.....	18
8.3 Qualités de conception.....	18
8.4 Scénario de démonstration.....	19
8.5 Intérêt en situation réelle.....	20
9. Difficultés rencontrées et solutions.....	21
10. Sécurité et bonnes pratiques.....	22
11. Compétences mobilisées.....	23
12. Limites et perspectives.....	24
12.1 Limites actuelles.....	24
12.2 Perspectives.....	24
13. Conclusion.....	25
Annexe A — Commandes essentielles.....	26
Annexe B — Configuration commentée du routeur frontière R3.....	27
Annexe C — Glossaire.....	28

1. Synthèse

Ce projet a consisté à **reconstruire, sur un simple ordinateur portable, le réseau informatique de deux organisations qui communiquent entre elles**, puis à **développer des outils qui surveillent ce réseau automatiquement**. Tout fonctionne dans des équipements virtuels : aucun matériel réseau physique n'a été acheté ni utilisé.

Concrètement, cinq routeurs virtuels (les équipements qui aigüillent les données d'un point à un autre) et deux ordinateurs clients ont été créés. Les routeurs utilisent les mêmes protocoles que ceux qui font fonctionner les réseaux d'entreprise et Internet : **OSPF**, pour s'organiser à l'intérieur d'une organisation, et **BGP**, pour échanger avec l'organisation voisine.

Dans un second temps, trois programmes en Python ont été écrits pour se connecter à ces routeurs et répondre automatiquement à trois questions que se pose tout administrateur réseau :

- **Le réseau est-il en bonne santé ?** Tous les routeurs se parlent-ils comme prévu ?
- **Ai-je une sauvegarde de toutes les configurations ?**
- **Quelqu'un a-t-il modifié une configuration sans prévenir ?** C'est ce qu'on appelle une « dérive de configuration ».

Indicateur	Valeur
Équipements virtuels déployés	7 (5 routeurs + 2 postes clients)
Organisations simulées (systèmes autonomes)	2 (AS65001 et AS65002)
Liaisons réseau virtuelles	7
Outils d'automatisation développés	3 (santé, sauvegarde, détection de dérive)
Tests de fonctionnement réussis	100 % (routage, connectivité, détection de pannes)
Matériel nécessaire	Un PC portable (ASUS Zenbook), rien d'autre
Coût des logiciels	0 € (tous gratuits et open source)

À retenir

Le laboratoire fonctionne de bout en bout sur le poste de travail : les données partent d'un ordinateur client, traversent quatre routeurs et deux organisations, puis arrivent à destination par le chemin prévu.

Les outils d'automatisation détectent correctement une panne simulée et une modification non autorisée. Ils indiquent précisément quel routeur est touché et ce qui a changé.

2. Contexte et objectifs

2.1 Point de départ

Ce projet s'inscrit dans un parcours de formation en cybersécurité (stage en cybersécurité, préparation de la certification Cisco CCNA). La théorie du routage s'apprend dans les cours, mais **la pratiquer demande normalement du matériel coûteux** : routeurs, commutateurs, câbles, ou un serveur puissant pour les simuler.

La seule ressource disponible étant un ordinateur portable, la question de départ était la suivante : **peut-on construire un réseau réaliste, utilisant les vrais protocoles de l'industrie, sans rien d'autre qu'un PC portable ?**

2.2 Objectifs

#	Objectif	Critère de réussite
O1	Pratiquer le routage dynamique avec les protocoles réels (OSPF, BGP)	Les routeurs échangent leurs routes et les données circulent de bout en bout
O2	Disposer d'un environnement reproductible	Le laboratoire se recrée à l'identique en une commande
O3	Automatiser la surveillance du réseau	Des scripts détectent seuls pannes et modifications
O4	Constituer une réalisation présentable (portfolio, recherche d'alternance)	Projet documenté, versionnable, démontrable en direct

2.3 Contraintes

- **Matériel** : un PC portable Windows, sans serveur ni équipement réseau.
- **Budget** : nul. Uniquement des logiciels gratuits.
- **Licences** : pas d'images de routeurs propriétaires (Cisco, Fortinet), dont l'usage hors cadre commercial est restreint.
- **Performance** : le laboratoire ne doit pas saturer la mémoire du portable.

2.4 Démarche et outils d'assistance

Le projet a été mené avec l'assistance d'outils d'intelligence artificielle, utilisés de manière transparente et encadrée :

- **Claude (assistant conversationnel)** : proposition et classement des projets possibles, conception de l'architecture, rédaction des configurations et des scripts, **validation préalable dans un environnement de test isolé**, puis accompagnement pas à pas lors de l'installation.
- **Claude Code (assistant exécutant des commandes sur le poste)** : déploiement du laboratoire sur la machine réelle, exécution des tests et vérification des résultats.
- **Rôle de l'auteur** : choix du projet et des orientations, installation de l'environnement (WSL2, Docker, containerlab), exécution des commandes nécessitant les droits administrateur, arbitrage des décisions, vérification manuelle du laboratoire.

Point d'attention

Les commandes qui modifient le système avec les droits administrateur (**sudo**) ont toutes été exécutées **par l'auteur lui-même**, après lecture. L'assistant a d'ailleurs refusé de désactiver la demande de mot de passe administrateur, pour ne pas affaiblir la sécurité du poste.

3. Les notions clés, expliquées simplement

Cette partie donne les bases nécessaires pour comprendre la suite, sans connaissances préalables. Chaque notion est suivie d'une comparaison avec la vie courante.

3.1 Réseau, adresse IP et sous-réseau

Un **réseau informatique** est un ensemble d'équipements reliés qui s'échangent des données. Pour que chaque donnée arrive au bon endroit, chaque équipement possède une **adresse IP**, par exemple 192.168.1.10.

Les adresses sont regroupées en **sous-réseaux**, notés avec une barre oblique : 192.168.1.0/24 désigne un groupe de 254 adresses utilisables, 10.1.12.0/30 un groupe de seulement 2 adresses. Un /30 est idéal pour relier exactement deux routeurs entre eux.

En clair

Une adresse IP est comme une **adresse postale**. Un sous-réseau /24 est un **quartier** de 254 maisons. Un /30 est une **rue qui ne contient que deux maisons**, face à face : parfait pour relier deux voisins.

3.2 Routeur et table de routage

Un **routeur** est l'équipement qui relie plusieurs réseaux et décide par où envoyer chaque donnée. Pour décider, il consulte sa **table de routage** : une liste qui indique, pour chaque destination, par quel voisin passer.

En clair

Le routeur est un **aiguilleur de gare**. Sa table de routage est le **tableau des correspondances** : « pour aller vers Lyon, quai 3 ; pour aller vers Lille, quai 7 ».

3.3 Routage dynamique et OSPF

Remplir les tables de routage à la main serait long et fragile : si un câble est coupé, il faudrait tout corriger manuellement. Les routeurs utilisent donc des **protocoles de routage dynamique** : ils discutent entre eux et construisent seuls leurs tables.

OSPF (Open Shortest Path First) est le protocole le plus répandu à l'intérieur des entreprises. Chaque routeur décrit ses liaisons à ses voisins. Tous finissent par avoir la même carte complète du réseau, et chacun calcule le chemin le plus court vers chaque destination. Chaque liaison a un **coût** : plus il est faible, plus le chemin est préféré.

Deux routeurs qui se sont reconnus et ont synchronisé leur carte sont dits **voisins en état « Full »**. Quand une liaison tombe, les routeurs recalculent un nouveau chemin : c'est la **convergence**.

En clair

OSPF fonctionne comme un **GPS partagé** entre tous les routeurs d'une entreprise. Chacun signale les routes qu'il voit. Tout le monde a la même carte, et en cas de « route barrée », le GPS recalcule automatiquement un itinéraire.

3.4 BGP et systèmes autonomes

Internet est composé de milliers d'organisations indépendantes (opérateurs, grandes entreprises, hébergeurs). Chacune gère son propre réseau, appelé **système autonome** (AS) et identifié par un numéro, par exemple AS65001.

Entre ces organisations, on n'utilise pas OSPF mais **BGP** (Border Gateway Protocol). BGP ne cherche pas seulement le chemin le plus court : il applique des **politiques**. Chaque organisation choisit quelles routes elle accepte de recevoir et lesquelles elle accepte d'annoncer. Quand BGP est utilisé entre deux organisations différentes, on parle d'**eBGP** (BGP externe).

En clair

Si OSPF est le GPS à l'intérieur d'un pays, BGP est la **diplomatie entre les pays** : chaque pays décide quels passages de frontière il ouvre, et à qui.

Une erreur de politique BGP peut rendre un site entier injoignable. C'est pourquoi le filtrage des annonces est essentiel, et il a été mis en place dans ce laboratoire.

3.5 Virtualisation, WSL2 et conteneurs

La **virtualisation** consiste à faire tourner des ordinateurs « logiciels » à l'intérieur d'un ordinateur physique. **WSL2** (Windows Subsystem for Linux) est une fonction de Windows qui fait tourner un vrai système Linux (ici Ubuntu) dans une petite machine virtuelle très légère, gérée par Windows lui-même.

Un **conteneur** va encore plus loin dans la légèreté : au lieu de simuler un ordinateur complet, il isole simplement un programme et ce dont il a besoin. **Docker** est l'outil standard pour créer et faire tourner ces conteneurs. Un conteneur démarre en une seconde et consomme très peu de mémoire.

En clair

Une machine virtuelle classique, c'est **construire une maison entière** pour chaque équipement. Un conteneur, c'est **un appartement dans un immeuble** : chacun a son espace privé, mais les fondations et la plomberie sont partagées. On peut en loger beaucoup plus sur le même terrain.

3.6 containerlab et FRRouting

FRRouting (FRR) est un logiciel de routeur libre et gratuit, utilisé en production par de grands acteurs du réseau. Installé dans un conteneur, il transforme ce conteneur en véritable routeur parlant OSPF et BGP.

containerlab est l'outil qui construit le laboratoire. On lui donne un fichier décrivant les équipements et leurs câbles (le fichier `lab.clab.yml`), et il crée tout automatiquement : les conteneurs, les câbles virtuels entre eux et un réseau d'administration.

En clair

containerlab est un **entrepreneur** qui construit un quartier entier à partir d'un **plan d'architecte**. Le plan (fichier YAML) décrit les bâtiments et les routes ; l'entrepreneur livre le quartier prêt à l'emploi en quelques secondes, et peut le reconstruire à l'identique à tout moment.

3.7 SSH, automatisation et dérive de configuration

SSH est le moyen standard et sécurisé de se connecter à distance à un équipement pour lui envoyer des commandes. **Netmiko** est une bibliothèque Python qui permet à un programme de se connecter en SSH à des équipements réseau, comme le ferait un administrateur, mais automatiquement et sur des dizaines d'équipements à la fois.

La **configuration** d'un routeur est l'ensemble de ses réglages. On en garde une version de référence validée, appelée **baseline** ou **configuration de référence**. Si la configuration réelle s'en écarte (modification manuelle oubliée, erreur, acte malveillant), on parle de **dérive de configuration**.

En clair

La baseline est une **photo de référence** d'une pièce bien rangée. Le détecteur de dérive reprend une photo chaque jour et entoure en rouge tout ce qui a bougé depuis.

4. Architecture du laboratoire

4.1 Vue d'ensemble technique

Le laboratoire repose sur un empilement de quatre couches logicielles. Chaque couche s'appuie sur celle du dessous :

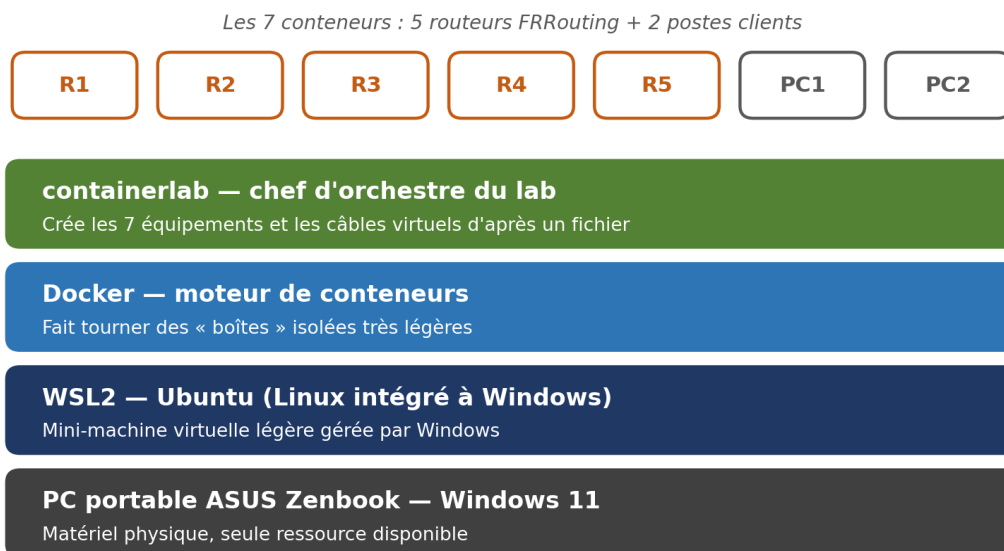


Figure 1 — Les couches techniques, du matériel aux équipements virtuels

Couche	Version utilisée	Rôle
Windows 11 (ASUS Zenbook)	—	Système hôte, seule ressource matérielle
WSL2 + Ubuntu	Ubuntu 26.04 LTS	Environnement Linux ; 16 cœurs virtuels et 15 Go de mémoire disponibles
Docker Engine	29.8.1	Moteur de conteneurs
containerlab	0.79.0	Construction du laboratoire à partir du fichier de topologie
FRRouting	10.2.1	Logiciel de routage des 5 routeurs
Python + Netmiko	Python 3.14, Netmiko 4.8.0	Scripts d'automatisation (phase 2)

4.2 Topologie du réseau

Le réseau simule **deux organisations** reliées par une seule liaison frontière :

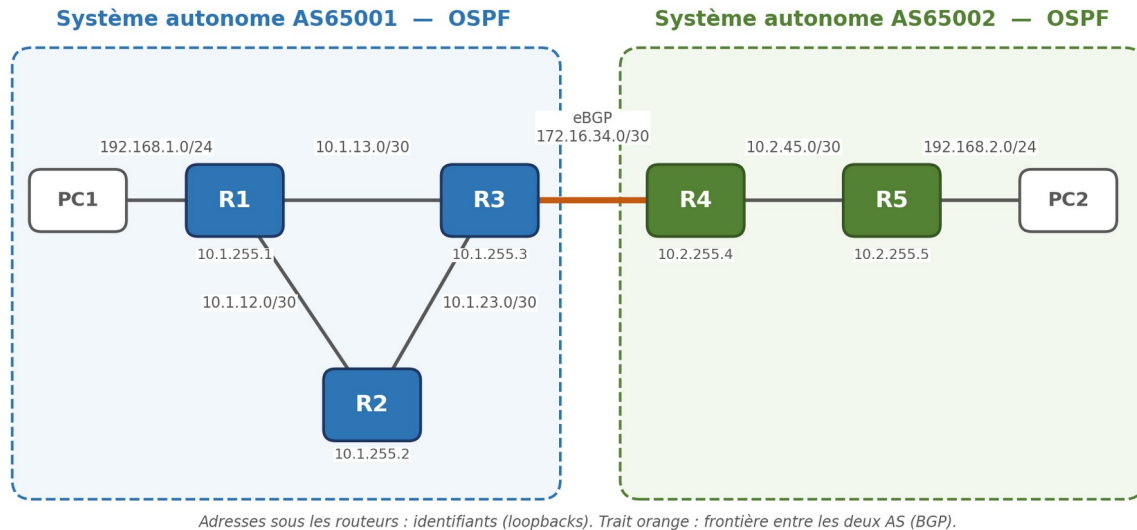


Figure 2 — Topologie du laboratoire : deux systèmes autonomes reliés par eBGP

- **Organisation 1 (AS65001)** : trois routeurs R1, R2 et R3 disposés en triangle, et un poste client PC1. Le triangle crée **deux chemins possibles** entre R1 et R3 : c'est ce qui permet de tester la bascule automatique en cas de panne.
- **Organisation 2 (AS65002)** : deux routeurs R4 et R5 en ligne, et un poste client PC2.
- **Frontière** : la liaison R3–R4 est la seule connexion entre les deux organisations. R3 et R4 y dialoguent en eBGP.

4.3 Plan d'adressage

Chaque liaison reçoit son propre sous-réseau. Le deuxième nombre de l'adresse identifie l'organisation (10.1.x.x pour AS65001, 10.2.x.x pour AS65002), ce qui rend le plan lisible d'un coup d'œil.

Liaison	Sous-réseau	Adresses	Usage
R1 ↔ R2	10.1.12.0/30	R1 .1 / R2 .2	Interne AS65001
R1 ↔ R3	10.1.13.0/30	R1 .1 / R3 .2	Interne AS65001
R2 ↔ R3	10.1.23.0/30	R2 .1 / R3 .2	Interne AS65001
R3 ↔ R4	172.16.34.0/30	R3 .1 / R4 .2	Frontière eBGP
R4 ↔ R5	10.2.45.0/30	R4 .1 / R5 .2	Interne AS65002
LAN de PC1	192.168.1.0/24	R1 .1 / PC1 .10	Réseau utilisateurs 1
LAN de PC2	192.168.2.0/24	R5 .1 / PC2 .10	Réseau utilisateurs 2
Identifiants AS65001	10.1.255.x/32	R1 .1, R2 .2, R3 .3	Adresses « loopback »
Identifiants AS65002	10.2.255.x/32	R4 .4, R5 .5	Adresses « loopback »
Administration	172.20.20.0/24	R1 .11 ... R5 .15	Accès SSH des scripts

En clair

Une adresse **loopback** est une adresse interne au routeur, qui ne dépend d'aucun câble. Elle sert de **carte d'identité** stable : même si une liaison tombe, le routeur reste joignable par cette adresse.

4.4 Logique de routage et justification des choix

Choix de configuration	Ce que cela fait	Pourquoi
OSPF en mode « point-to-point » sur les liaisons entre routeurs	Les deux routeurs deviennent voisins immédiatement	Sur une liaison à deux équipements, l'élection d'un routeur « chef » (DR/BDR) est inutile et ralentit la mise en route
Interfaces « passives » (réseaux clients, loopbacks, frontière)	Le réseau est annoncé, mais aucun dialogue OSPF n'y est engagé	Sécurité : on n'échange pas d'informations de routage avec les postes clients ni avec l'organisation voisine
Filtrage BGP par listes de préfixes et route-maps, dans les deux sens	Chaque organisation n'annonce que ses propres réseaux et n'accepte que ceux attendus du voisin	Évite les fuites de routes ; FRR refuse d'ailleurs par défaut d'échanger des routes eBGP sans politique
Agrégat /16 adossé à une route « Null0 »	Chaque organisation annonce un seul bloc global (10.1.0.0/16) au lieu de chaque petit réseau	Pratique standard des opérateurs : annonces stables et tables de routage plus petites
Redistribution de BGP dans OSPF sur R3 et R4	Les routes apprises de l'autre organisation sont diffusées à tous les routeurs internes	Sans cela, R1 et R2 ne sauraient pas comment joindre l'organisation voisine

4.5 Organisation des fichiers du projet

Fichier ou dossier	Rôle
lab.clab.yml	Plan du laboratoire : équipements, adresses d'administration, câbles
docker/Dockerfile	Recette de l'image des routeurs : FRR 10.2.1 + serveur SSH + compte netops
configs/daemons	Liste des services de routage activés (OSPF, BGP, BFD)
configs/r1 ... r5/frr.conf	Configuration de chaque routeur, chargée au démarrage
automation/inventory.yml	Inventaire : adresse de chaque routeur et état attendu (voisins OSPF, sessions BGP)
automation/labtools.py	Briques communes : connexion SSH, exécution des commandes, parallélisme
automation/health.py	Contrôle de santé du routage
automation/backup.py	Sauvegarde des configurations
automation/drift.py	Détection de dérive de configuration
README.md	Guide pas à pas : installation, vérifications, exercices, dépannage

5. Choix techniques et alternatives étudiées

Pour chaque brique du projet, plusieurs solutions existaient. Le tableau ci-dessous résume les options comparées et la raison du choix retenu.

5.1 Plateforme d'exécution

Option	Avantages	Inconvénients	Verdict
WSL2 + Ubuntu	Très léger, mémoire rendue à Windows quand elle est libre, pas de configuration réseau complexe	Nécessite une installation initiale avec droits administrateur	Retenu
Machine virtuelle VirtualBox	Instantanés (retour arrière faciles), isolation forte	Mémoire bloquée en permanence ; cohabite mal avec l'hyperviseur de Windows, performances réduites	Écarté
Double démarrage Linux	Performances maximales	Redémarrage obligatoire pour changer de système, risque sur le disque	Écarté

5.2 Outil de simulation réseau

Option	Avantages	Inconvénients	Verdict
containerlab	Laboratoire décrit dans un fichier texte (reproductible, versionnable), démarrage en secondes, très faible consommation	Uniquement en ligne de commande	Retenu
GNS3 / EVE-NG	Interface graphique, images de nombreux constructeurs	Chaque routeur virtuel consomme 0,5 à 2 Go de mémoire : un portable sature vite	Écarté
Cisco Packet Tracer	Parfait pour le CCNA, interface simple	Simulateur (pas un vrai logiciel de routage), impossible à automatiser en Python	Complémentaire

5.3 Logiciel des routeurs

Option	Avantages	Inconvénients	Verdict
FRRouting	Gratuit, libre, utilisé en production, très léger en conteneur, syntaxe proche de Cisco	Quelques différences de syntaxe avec Cisco IOS	Retenu
Cisco IOSv / CSR	Syntaxe Cisco exacte	Licence restrictive, lourd (machine virtuelle complète)	Écarté
VyOS	Libre, complet	Syntaxe éloignée de celle enseignée au CCNA	Écarté

5.4 Bibliothèque d'automatisation

Option	Avantages	Inconvénients	Verdict
Netmiko	Simple, standard du métier, compatible avec des dizaines de constructeurs (Cisco, Fortinet, Juniper...)	Travaille au niveau des commandes texte	Retenu
NAPALM	Interface unifiée entre constructeurs	Pas de pilote officiel pour FRR	Écarté
Ansible	Très répandu en entreprise	Couche supplémentaire à apprendre ; moins formateur pour comprendre les mécanismes	Évolution possible

6. Déroulement du projet

Le projet s'est déroulé en cinq étapes, entre la conception et la vérification finale sur le poste de travail.

Étape	Contenu	Résultat
1. Conception	Choix du projet, topologie, plan d'adressage, configurations, scripts	Projet complet (16 fichiers) et guide d'utilisation
2. Validation préalable	Reproduction du réseau dans un environnement de test isolé	Configurations et scripts validés ; 1 anomalie corrigée
3. Préparation du poste	Installation de WSL2, Docker, containerlab et Python	Environnement opérationnel après résolution de 6 incidents
4. Déploiement réel	Construction de l'image, lancement du laboratoire, tests des phases 1 et 2	100 % des vérifications réussies
5. Vérification manuelle	Remise à zéro, contrôle de santé, exploration d'un routeur	Laboratoire propre et conforme

6.1 Étape 1 : conception

Le projet a été choisi parmi huit idées classées selon leur intérêt pédagogique, leur valeur pour une recherche d'alternance et leur faisabilité sur un portable. Le couple « laboratoire de routage en conteneurs » puis « automatisation » a été retenu, car il combine réseau et programmation, ce qui est particulièrement recherché (profil « NetDevOps »).

Les cinq fichiers de configuration des routeurs n'ont pas été écrits un par un : ils ont été **générés par un petit programme à partir d'un plan d'adressage unique**. Cette méthode évite les incohérences, comme une même adresse attribuée deux fois ou une faute de frappe sur un routeur.

6.2 Étape 2 : validation préalable en environnement de test

Avant de livrer le projet, il a été testé dans l'environnement de travail de l'assistant. Celui-ci ne pouvait pas télécharger les images Docker (accès aux registres bloqué). Le réseau a donc été **reproduit à l'identique avec les « espaces de noms réseau » de Linux**, la technologie sur laquelle reposent les conteneurs eux-mêmes, et avec FRR 8.4 installé localement.

Cette réplique a permis de vérifier la convergence d'OSPF et de BGP, la connectivité de bout en bout, puis le fonctionnement des trois scripts en SSH, y compris la détection d'une panne et d'une dérive volontairement provoquées.

Anomalie détectée et corrigée

Lors du premier test, les trois scripts signalaient **tous les routeurs en erreur**, alors que le réseau fonctionnait. La cause : le compte non-administrateur utilisé par les scripts ne pouvait pas lire un fichier de réglages facultatif (`vtysh.conf`). Le logiciel affichait alors un simple avertissement, que les scripts interprétaient à tort comme une erreur fatale.

Correction en deux temps : les scripts ignorent désormais cet avertissement sans conséquence, et la recette de l'image rend le fichier lisible. Sans cette validation préalable, le problème serait apparu directement sur le poste de travail.

En clair

C'est comme une alarme qui se déclenche parce qu'une porte de placard est entrouverte : le signal était réel, mais il ne signalait aucun danger. On a appris à l'alarme à ignorer ce placard, et on a fermé la porte.

6.3 Étape 3 : préparation du poste de travail

L'installation a été réalisée par l'auteur, guidé étape par étape. Six incidents ont été rencontrés et résolus (détaillés au chapitre 9) :

- **Téléchargement d'Ubuntu figé à 24,7 %** : la fenêtre PowerShell était passée en mode « Sélection » après un clic, ce qui met le programme en pause. L'installation a ensuite été relancée avec l'option `--web-download`, qui télécharge directement la distribution.
- **Deux distributions Ubuntu installées** : une distribution « Ubuntu » existait déjà, et le projet y avait été copié. La nouvelle « Ubuntu-24.04 » a donc été laissée de côté, et « Ubuntu » définie comme distribution par défaut.
- **Messages d'avertissement pendant l'installation de Docker et au démarrage de WSL** : ils ont été analysés et jugés non bloquants.
- **Droits d'utilisation** : l'utilisateur a été ajouté aux groupes `docker` et `clab_admins`, pour utiliser Docker et containerlab sans droits administrateur.
- **Module Python manquant** : le paquet `python3.14-venv`, nécessaire pour créer l'environnement Python isolé, a été installé.

6.4 Étape 4 : déploiement réel et tests

Une fois l'environnement prêt, Claude Code a exécuté le déploiement sur le poste :

1. Construction de l'image des routeurs (`frr-ssh:10.2.1`) : réussie sans erreur.
2. Déploiement du laboratoire : les 7 conteneurs démarrés.
3. Vérification du routage après environ 30 secondes : voisins OSPF « Full », session BGP établie, routes conformes.
4. Test de bout en bout : aucune perte de paquets, chemin exact.
5. Création de l'environnement Python et installation de Netmiko 4.8.0.
6. Scénario complet de la phase 2 : santé, sauvegarde, panne simulée, détection.

Point important : c'était la **première exécution avec la vraie image FRR 10.2.1** (la validation préalable utilisait FRR 8.4). Aucune adaptation n'a été nécessaire, ce qui confirme que les deux versions sont compatibles sur tout ce qui est utilisé ici.

6.5 Étape 5 : remise à zéro et vérification manuelle

La démonstration de la phase 2 avait volontairement laissé deux pannes actives. Le laboratoire a été entièrement reconstruit avec la commande `containerlab deploy --reconfigure` (7 équipements recréés en environ 3 secondes). Un contrôle de santé a confirmé le retour à la normale, puis l'auteur a exploré manuellement le routeur R1.

7. Résultats : le routage (phase 1)

Les résultats ci-dessous proviennent du laboratoire réel, sur le poste de travail.

7.1 Voisinages OSPF

Sur R1, la commande `show ip ospf neighbor` montre deux voisins, R2 et R3, en état **Full/-**, c'est-à-dire parfaitement synchronisés. Le tiret indique qu'aucune élection de routeur « chef » n'a lieu, ce qui est le comportement attendu sur des liaisons point à point.

```
r1# show ip ospf neighbor
Neighbor ID    Pri  State           Up Time   Dead Time   Address     Interface
10.1.255.2      1  Full/-         2m02s    33.043s    10.1.12.2   eth1:10.1.12.1
10.1.255.3      1  Full/-         2m07s    33.000s    10.1.13.2   eth2:10.1.13.1
```

7.2 Table de routage de R1

La table de routage de R1 montre comment il a appris chaque destination. Sélection commentée :

Destination	Source	Passe par	Signification
10.1.23.0/30	OSPF	R2 et R3	Deux chemins de coût égal (20) : le trafic est réparti entre les deux (ECMP, partage de charge)
10.1.255.2/32	OSPF	R2	Identifiant de R2, appris via OSPF
10.2.0.0/16	OSPF (externe)	R3	Réseaux de l' organisation voisine , appris par R3 en BGP puis rediffusés dans OSPF
192.168.2.0/24	OSPF (externe)	R3	Réseau de PC2, de l'autre côté de la frontière
172.16.34.0/30	OSPF	R3	Liaison frontière, annoncée en passif
192.168.1.0/24	Connecté	eth3	Réseau de PC1, directement branché sur R1

En clair

R1 n'a jamais parlé directement à l'organisation voisine. Il connaît pourtant le chemin vers PC2 : R3 l'a appris par la « diplomatie » BGP, puis l'a relayé à ses collègues par le « GPS » OSPF. C'est exactement ce qui se passe dans une entreprise connectée à un opérateur.

7.3 Session BGP entre les deux organisations

Routeur	Voisin BGP	État	Préfixes reçus	Préfixes envoyés
R3 (AS65001)	172.16.34.2 (R4)	Established	2	2
R4 (AS65002)	172.16.34.1 (R3)	Established	2	2

Chaque organisation annonce exactement deux préfixes, son agrégat /16 et son réseau client /24, et en reçoit exactement deux. Les filtres fonctionnent : rien d'autre ne passe la frontière.

7.4 Test de bout en bout

Le test final consiste à envoyer des données de PC1 vers PC2 et à tracer leur parcours avec la commande `traceroute`. Résultat : **0 % de perte**, et le chemin suivi est exactement celui prévu.

Saut	Adresse	Équipement	Commentaire
1	192.168.1.1	R1	Passerelle de PC1
2	10.1.13.2	R3	Chemin direct R1 → R3 (coût le plus faible)
3	172.16.34.2	R4	Passage de la frontière entre organisations
4	10.2.45.2	R5	Routeur de sortie vers PC2
5	192.168.2.10	PC2	Destination atteinte

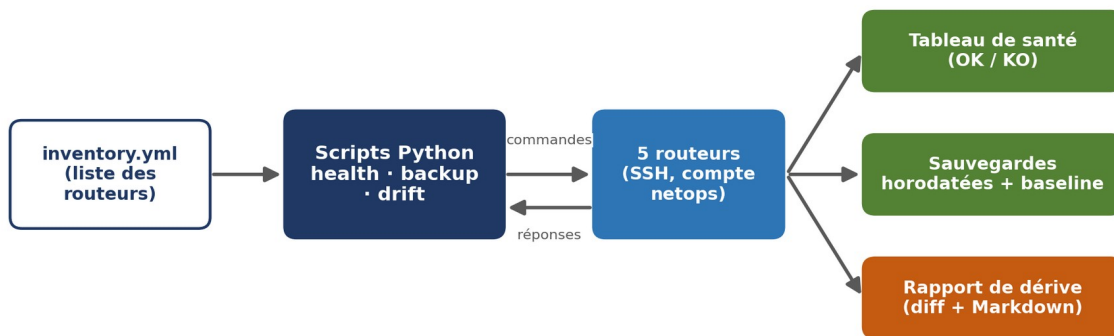
À retenir

La phase 1 est validée : les deux protocoles fonctionnent, les politiques de filtrage sont respectées et la connectivité entre les deux organisations est totale.

8. Résultats : l'automatisation (phase 2)

8.1 Principe de fonctionnement

Les trois scripts suivent le même schéma : ils lisent l'inventaire, se connectent en SSH à chaque routeur **en parallèle**, envoient des commandes, analysent les réponses et produisent un résultat lisible.



Les 5 connexions partent en parallèle : un routeur en panne ne bloque pas les autres.

Figure 3 — Chaîne d'automatisation : de l'inventaire aux rapports

8.2 Les trois outils

Outil	Question à laquelle il répond	Ce qu'il produit	Code retour
health.py	Le routage est-il dans l'état attendu ?	Tableau par routeur : voisins OSPF, sessions BGP, verdict OK/KO	0 = tout va bien 1 = problème
backup.py	Ai-je une copie de toutes les configurations ?	Un dossier daté par exécution ; avec -- baseline , fige la référence	0 = réussi 1 = échec partiel
drift.py	Quelqu'un a-t-il modifié une configuration ?	Différences ligne par ligne avec la référence, rapport Markdown en option	0 = conforme 1 = dérive 2 = erreur

En clair

Le **code retour** est un chiffre que chaque programme renvoie en se terminant. Il permet à d'autres programmes de réagir automatiquement, par exemple envoyer une alerte uniquement si **drift.py** renvoie 1. C'est ce qui rend ces outils intégrables dans une chaîne de surveillance.

8.3 Qualités de conception

- **Moindre privilège** : les scripts se connectent avec un compte dédié, **netops**, qui peut lire l'état du routage mais n'a pas les droits d'administrateur du routeur.
- **Parallélisme** : les cinq routeurs sont interrogés simultanément. Sur 50 routeurs, le gain de temps serait considérable.
- **Tolérance aux pannes** : un routeur injoignable est signalé comme tel, sans interrompre le traitement des autres.

- **Lecture de données structurées** : le contrôle de santé exploite les sorties au format JSON de FRR plutôt que du texte brut. C'est plus fiable, car le résultat ne dépend pas de la mise en page de l'affichage.
- **Comparaison intelligente** : la détection de dérive ignore les lignes qui changent sans modification réelle (numéro de version, bannières), pour éviter les fausses alertes.

8.4 Scénario de démonstration

Pour prouver l'efficacité des outils, deux incidents ont été volontairement provoqués après avoir figé la configuration de référence :

Incident simulé	Commande utilisée	Conséquence réelle
Modification « sauvage » du coût OSPF sur R2	<code>ip ospf cost 50</code> sur l'interface eth1	Aucune panne visible, mais la configuration n'est plus conforme
Coupure administrative de la session BGP sur R4	<code>neighbor 172.16.34.1 shutdown</code>	Les deux organisations ne se voient plus : PC1 ne joint plus PC2

Détection par le contrôle de santé

Routeur	OSPF	Full	BGP	Verdict
r1	2/2	-	-	OK
r2	2/2	-	-	OK
r3	2/2	-	172.16.34.2 Active (0 pfx)	KO : BGP 172.16.34.2 Active
r4	1/1	-	172.16.34.1 Idle (Admin) (0 pfx)	KO : BGP 172.16.34.1 Idle (Admin)
r5	1/1	-	-	OK

Le contrôle identifie immédiatement les deux routeurs concernés. Il distingue même les deux côtés du problème : R4 est en « Idle (Admin) », c'est-à-dire coupé volontairement par un administrateur, tandis que R3 est en « Active » et essaie de se reconnecter sans succès. Le coût OSPF modifié n'apparaît pas ici, car il ne casse rien : c'est justement le rôle du second outil.

Détection par l'analyse de dérive

r1	CONFORME
r2	DÉRIVE
r3	CONFORME
r4	DÉRIVE
r5	CONFORME
<pre> === r2 === ip address 10.1.12.2/30 ip ospf area 0 + ip ospf cost 50 ip ospf network point-to-point === r4 === neighbor 172.16.34.1 description r3-AS65001 + neighbor 172.16.34.1 shutdown </pre>	

Les lignes précédées d'un « + » sont celles ajoutées par rapport à la référence. L'outil montre **exactement** les deux modifications, avec leur contexte, sans aucune fausse alerte sur les trois autres routeurs.

À retenir

Les deux outils sont complémentaires : `health.py` détecte ce qui est **cassé**, `drift.py` détecte ce qui a **changé**, même quand rien n'est encore cassé. Une modification non autorisée qui ne provoque pas de panne immédiate reste un risque, et en cybersécurité, c'est souvent la trace d'une intrusion.

8.5 Intérêt en situation réelle

Besoin en entreprise	Apport des outils
Vérifier le réseau après une intervention planifiée	Contrôle de santé en quelques secondes au lieu de connexions manuelles une à une
Pouvoir restaurer une configuration après un incident	Sauvegardes datées et automatiques
Audit de sécurité et conformité	Preuve écrite que les équipements sont conformes à la référence validée
Détection d'intrusion ou d'erreur humaine	Toute modification non prévue est repérée et documentée
Traçabilité	Rapports Markdown joignables à un ticket, historique versionnable avec Git

9. Difficultés rencontrées et solutions

Chaque difficulté a été analysée pour en comprendre la cause, et non simplement contournée. Le tableau résume les incidents et ce qu'ils ont appris.

#	Problème	Cause	Solution	Enseignement
1	Images Docker impossibles à télécharger dans l'environnement de conception	Accès aux registres bloqué dans cet environnement	Réplique du réseau avec les espaces de noms Linux et FRR installé localement	Comprendre la technologie sous-jacente permet de tester même sans l'outil final
2	Tous les routeurs signalés en erreur par les scripts	Avertissement bénin de vttysh interprété comme une erreur	Filtrage de l'avertissement + correction des droits dans l'image	Tester avec un compte non-administrateur, comme en production
3	Téléchargement d'Ubuntu figé	Fenêtre PowerShell en mode « Sélection » (pause)	Touche Échap, puis relance avec <code>--web-download</code>	Un clic dans une console Windows peut suspendre un programme
4	Deux distributions Ubuntu	Une distribution existait déjà avant l'installation	Choix de la distribution contenant le projet, définie par défaut	Toujours vérifier l'existant (<code>wsl -l -v</code>) avant d'installer
5	Message « utilisez Docker Desktop »	Simple recommandation du script d'installation	Ignoré : Docker Engine natif retenu, plus fiable avec containerlab	Lire un avertissement avant de le considérer comme une erreur
6	Avertissement systemd au démarrage de WSL	Session utilisateur systemd non démarrée	Non bloquant : Docker fonctionne (test hello-world réussi)	Valider par un test fonctionnel plutôt que par l'absence de message
7	containerlab demande les droits administrateur	Comportement par défaut de l'outil	Ajout au groupe <code>clab_admins</code>	Préférer des droits ciblés à un accès administrateur global
8	Création de l'environnement Python impossible	Paquet <code>python3.14-venv</code> absent	Installation du paquet	Les dépendances système ne sont pas couvertes par pip
9	L'assistant ne peut pas saisir le mot de passe administrateur	Mesure de sécurité volontaire	Commandes sensibles exécutées par l'auteur ; demande de désactivation du mot de passe refusée	L'automatisation ne doit pas affaiblir la sécurité du poste
10	Pannes de démonstration toujours actives	Le scénario de démonstration ne les annulait pas	Reconstruction complète avec <code>--reconfigure</code>	Un laboratoire décrit dans un fichier se remet à zéro en secondes

10. Sécurité et bonnes pratiques

S'agissant d'un projet mené dans un cadre de cybersécurité, plusieurs règles ont été appliquées volontairement.

Principe	Application dans le projet
Moindre privilège	Compte netops non-administrateur pour les scripts ; groupes docker et clab_admins plutôt que des droits administrateur globaux
Pas d'affaiblissement du poste	Refus de désactiver la demande de mot de passe administrateur ; commandes sensibles lues puis exécutées par l'utilisateur
Séparation des réseaux	Réseau d'administration (172.20.20.0/24) distinct des réseaux de production simulés
Filtrage aux frontières	Politiques BGP explicites dans les deux sens ; interfaces passives vers les clients et le voisin
Identifiants de laboratoire	Mot de passe volontairement simple (netops/netops), strictement limité au laboratoire et documenté comme tel
Reproductibilité et traçabilité	Infrastructure décrite dans des fichiers texte, sauvegardes datées, projet prêt pour Git

Point d'attention

Les outils développés fonctionneraient aussi sur de vrais équipements (Cisco, Fortinet...). **Ils ne doivent jamais être utilisés sur un réseau d'entreprise sans autorisation écrite préalable**, même en lecture seule : une connexion SSH à un équipement de production est un acte encadré.

Les identifiants du laboratoire devraient être remplacés par une authentification par clé SSH et un coffre-fort de secrets avant tout usage réel.

11. Compétences mobilisées

Domaine	Compétences mises en œuvre	Lien avec la formation ou le métier
Routage IP	Plan d'adressage, sous-réseaux, tables de routage, partage de charge (ECMP)	Cœur du programme CCNA
OSPF	Voisinages, types de réseau, interfaces passives, coûts, redistribution	CCNA (OSPF à une seule zone)
BGP	Systèmes autonomes, eBGP, listes de préfixes, route-maps, agrégation	Au-delà du CCNA : niveau CCNP / opérateur
Linux et virtualisation	WSL2, gestion des paquets, groupes et droits, espaces de noms réseau	Administration système
Conteneurs	Docker, construction d'images, containerlab, infrastructure décrite en fichiers	DevOps, infrastructure as code
Programmation Python	SSH automatisé (Netmiko), parallélisme, lecture de JSON, comparaison de textes, codes retour	NetDevOps, automatisation
Cybersécurité	Moindre privilège, détection de modifications non autorisées, filtrage aux frontières	Stage et spécialisation en cybersécurité
Méthode	Validation avant livraison, diagnostic d'incidents, documentation	Tout métier technique

12. Limites et perspectives

12.1 Limites actuelles

- **Syntaxe** : FRR est proche de Cisco IOS, sans y être identique. Quelques commandes diffèrent de celles de l'examen CCNA.
- **Périmètre** : IPv4 uniquement, une seule zone OSPF, une seule liaison entre les organisations (pas de redondance BGP).
- **Échelle** : 5 routeurs. Le comportement sur des centaines d'équipements n'a pas été éprouvé.
- **Authentification** : mot de passe simple, acceptable pour un laboratoire mais pas au-delà.
- **Usage de l'IA** : une large part de la configuration et du code a été produite avec assistance. La maîtrise réelle se construit par les exercices et par l'écriture de nouveaux outils par l'auteur.

12.2 Perspectives

Priorité	Évolution	Intérêt
1	Réaliser les exercices du guide (convergence, ingénierie de trafic, politique BGP, BFD, capture Wireshark, double liaison)	Maîtrise approfondie des protocoles ; préparation CCNA
2	Développer un validateur de changement : capture de l'état du réseau avant et après une intervention, puis comparaison automatique	Outil réellement utile en entreprise, peu couvert par des solutions légères
3	Ajouter un audit de conformité fondé sur des règles (ex. « toute session eBGP doit être filtrée »), avec niveaux de sévérité	Prolongement direct du projet de stage d'audit de pare-feux
4	Publier le projet sur GitHub, avec schémas et démonstration	Visibilité pour la recherche d'alternance
5	Script de remédiation : remise automatique en conformité	Boucle complète détection → correction

13. Conclusion

Ce projet démontre qu'un **simple ordinateur portable suffit à construire un réseau multi-organisations réaliste**, utilisant les mêmes protocoles et le même logiciel de routage que des réseaux en production, sans matériel ni licence.

Les quatre objectifs fixés sont atteints. Le routage OSPF et BGP fonctionne de bout en bout (O1). Le laboratoire se reconstruit à l'identique en une commande (O2). Trois outils surveillent automatiquement sa santé, sauvegardent ses configurations et détectent toute modification (O3). Enfin, l'ensemble est documenté et démontrable (O4).

Au-delà du résultat technique, la démarche a été aussi importante que le produit : **validation avant livraison**, qui a permis de découvrir une anomalie avant qu'elle n'atteigne le poste de travail, **diagnostic méthodique** de chaque incident d'installation, et **refus des raccourcis qui affaiblissent la sécurité**.

Le laboratoire sert désormais de **banc d'essai** : un réseau que l'on peut casser sans risque, sur lequel développer et éprouver de nouveaux outils avant de les appliquer, avec autorisation, à de vrais réseaux.

Annexe A — Commandes essentielles

Toutes les commandes s'exécutent dans le terminal Ubuntu (WSL), depuis le dossier `~/lab-reseau-frr`.

Action	Commande
Ouvrir Ubuntu depuis PowerShell	<code>wsl -d Ubuntu</code>
Construire l'image des routeurs	<code>docker build -t frr-ssh:10.2.1 docker/</code>
Démarrer le laboratoire	<code>containerlab deploy -t lab.clab.yml</code>
Remettre le laboratoire à zéro	<code>containerlab deploy -t lab.clab.yml --reconfigure</code>
Arrêter et nettoyer	<code>containerlab destroy -t lab.clab.yml --cleanup</code>
Ouvrir la console d'un routeur	<code>docker exec -it clab-frr-lab-r1 vtysh</code>
Voir les voisins OSPF (dans vtysh)	<code>show ip ospf neighbor</code>
Voir la table de routage (dans vtysh)	<code>show ip route</code>
Voir l'état BGP (dans vtysh)	<code>show bgp ipv4 summary</code>
Tracer le chemin PC1 → PC2	<code>docker exec clab-frr-lab-pc1 traceroute -n 192.168.2.10</code>
Activer l'environnement Python	<code>cd automation && source .venv/bin/activate</code>
Contrôle de santé	<code>python health.py</code>
Sauvegarde + référence	<code>python backup.py --baseline</code>
Détection de dérive avec rapport	<code>python drift.py --report rapport.md</code>

Annexe B — Configuration commentée du routeur frontière R3

R3 est le routeur le plus complet : il participe à OSPF à l'intérieur de son organisation et à BGP vers l'organisation voisine. Les lignes précédées de « ! » sont des commentaires ajoutés pour ce rapport.

```
! --- Interface vers l'organisation voisine ---
interface eth3
  description eBGP-vers-r4-AS65002
  ip address 172.16.34.1/30
  ip ospf area 0
  ip ospf passive          ! annoncée dans OSPF, mais aucun dialogue OSPF avec le voisin
!
! --- OSPF : diffuse en interne les routes apprises par BGP ---
router ospf
  ospf router-id 10.1.255.3
  redistribute bgp
!
! --- Route de rejet : ancre l'agrégat annoncé en BGP ---
ip route 10.1.0.0/16 Null0
!
! --- Filtres : ce qu'on annonce (OUT) et ce qu'on accepte (IN) ---
ip prefix-list PL-EBGP-OUT seq 10 permit 10.1.0.0/16
ip prefix-list PL-EBGP-OUT seq 20 permit 192.168.1.0/24
ip prefix-list PL-EBGP-IN seq 10 permit 10.2.0.0/16
ip prefix-list PL-EBGP-IN seq 20 permit 192.168.2.0/24
route-map RM-EBGP-IN permit 10
  match ip address prefix-list PL-EBGP-IN
route-map RM-EBGP-OUT permit 10
  match ip address prefix-list PL-EBGP-OUT
!
! --- BGP : session avec R4 dans l'AS 65002, filtrée dans les deux sens ---
router bgp 65001
  bgp router-id 10.1.255.3
  neighbor 172.16.34.2 remote-as 65002
  address-family ipv4 unicast
    network 10.1.0.0/16
    network 192.168.1.0/24
  neighbor 172.16.34.2 route-map RM-EBGP-IN in
  neighbor 172.16.34.2 route-map RM-EBGP-OUT out
  exit-address-family
```

Annexe C — Glossaire

Terme	Définition
Adresse IP	Identifiant numérique d'un équipement sur un réseau (ex. 192.168.1.10)
Agrégat	Regroupement de plusieurs réseaux en une seule annonce plus large (ex. 10.1.0.0/16)
AS (système autonome)	Réseau d'une organisation, géré de façon indépendante, identifié par un numéro
Baseline	Configuration de référence validée, servant de point de comparaison
BFD	Protocole de détection très rapide de coupure de liaison
BGP / eBGP	Protocole de routage entre organisations ; eBGP désigne son usage entre deux AS différents
Code retour	Chiffre renvoyé par un programme en fin d'exécution (succès ou échec)
Conteneur	Environnement isolé et léger faisant tourner un programme et ses dépendances
containerlab	Outil qui construit un laboratoire réseau de conteneurs à partir d'un fichier de description
Convergence	Moment où tous les routeurs ont recalculé leurs routes après un changement
Dérive de configuration	Écart entre la configuration réelle d'un équipement et sa configuration de référence
Docker	Logiciel standard de création et d'exécution de conteneurs
ECMP	Répartition du trafic sur plusieurs chemins de même coût
FRRouting (FRR)	Logiciel de routage libre, utilisé en production, qui transforme un système Linux en routeur
Loopback	Adresse interne d'un routeur, indépendante des câbles, servant d'identifiant stable
Netmiko	Bibliothèque Python de connexion SSH automatisée aux équipements réseau
Null0	Destination « poubelle » : le trafic envoyé vers elle est détruit
OSPF	Protocole de routage interne qui calcule le plus court chemin à partir d'une carte partagée
Préfixe	Notation d'un réseau (ex. 192.168.1.0/24) ; en BGP, synonyme de route annoncée
Prefix-list / route-map	Filtres qui définissent quelles routes sont acceptées ou annoncées
Redistribution	Injection de routes apprises par un protocole dans un autre (ici BGP vers OSPF)
Routeur	Équipement qui relie des réseaux et oriente chaque donnée vers sa destination
SSH	Protocole de connexion à distance chiffrée
Sous-réseau (/24, /30)	Groupe d'adresses ; le nombre après la barre indique sa taille (/30 = 2 adresses utilisables)
Table de routage	Liste des destinations connues d'un routeur et du chemin pour les atteindre
Traceroute	Outil qui affiche la liste des routeurs traversés pour atteindre une destination
vttysh	Console de commande des routeurs FRR, similaire à celle des routeurs Cisco
WSL2	Fonction de Windows qui fait tourner un vrai système Linux